AWS Scenario-Based Interview Prep

DevOps · SRE · Cloud Engineer Roles

> **Format:** Every question is scenario-driven — production-like, tricky, and conceptual. **Topics:** Networking · EC2 · Databases · Security No theory-only questions. No definitions for their own sake.

Table of Contents

- Section 1: Networking
- Section 2: EC2
- Section 3: Databases
- Section 4: Security

Section 1: Networking

Scenario 1.1 — The Unreachable Private Instance

> **You deploy an EC2 instance in a private subnet. You can SSH into a bastion host in the public subnet just fine, but when you try to SSH from the bastion into the private instance, it times out. Both instances are in the same VPC. What do you investigate, and in what order?**

Answer

A timeout (not a rejection) means packets are going out but responses aren't returning — or packets aren't arriving at all. Work layer by layer:

Step 1 — Verify the private instance's Security Group

The private instance's SG must explicitly allow **inbound SSH (port 22)** from the bastion's private IP or, better, from the bastion's Security Group ID as the source. New SGs deny all inbound by default.

```
Inbound Rule on Private Instance SG:
  Type: SSH
  Port: 22
  Source: sg-xxxxxxx (bastion's SG)   ← preferred over IP
```

Step 2 — Check the NACL on the private subnet

NACLs are stateless. Even if inbound SSH is allowed, the NACL must also allow **outbound ephemeral ports (1024–65535)** back to the bastion. This is the most commonly missed step.

```
Private Subnet NACL:
  Inbound:  Allow TCP 22 from bastion subnet CIDR (e.g., 10.0.1.0/24)
  Outbound: Allow TCP 1024-65535 to bastion subnet CIDR
```

Also check the **public subnet's NACL** for the reverse direction (outbound to 22, inbound from 1024–65535).

Step 3 — Confirm the private instance has a private IP and the route table is sane

SSH from bastion should target the private IP of the instance. Verify there's no misconfigured route table sending traffic out via IGW (which it won't have).

Step 4 — Validate the bastion's outbound Security Group

By default SGs allow all outbound, but if someone has hardened the bastion's SG, port 22 outbound must be allowed to the private subnet CIDR.

Step 5 — Use VPC Flow Logs to confirm

Enable flow logs on the VPC or ENI level. Look for:

- `ACCEPT` on inbound to private instance → SG/NACL is fine, issue is OS-level (sshd not running, wrong key, firewalld)
- `REJECT` on inbound → SG or NACL is blocking
- No record at all → traffic never arrived, routing issue

Key Insights / Pitfalls

- **The #1 miss:** Forgetting NACL outbound ephemeral ports. Security Groups being stateful lulls people into forgetting NACLs are stateless.
- **Second most common miss:** Checking the inbound SG on the private instance but not the outbound SG on the bastion.
- **Timeout vs Refused:** A timeout usually points to network-level blocking (SG, NACL, routing). A "Connection refused" response means the instance is reachable but sshd is rejecting — now it's an OS-level problem.
- **Don't skip flow logs.** Guessing layers wastes time in production. Flow logs tell you exactly where traffic is being accepted or rejected.

Scenario 1.2 — Asymmetric Routing Breaks the VPN

> **Your team connects an on-premises data centre to AWS via a Site-to-Site VPN. Traffic from on-prem to EC2 works perfectly. But traffic initiated from EC2 back to on-prem drops intermittently. What could cause this, and how do you fix it?**

Answer

Asymmetric routing is the most likely culprit. This happens when traffic takes one path in one direction and a different path back, and stateful devices (firewalls, NAT) on one of those paths don't have state for the

return flow.

Root Cause Analysis:

Scenario A — Multiple VPN tunnels, traffic using different tunnels per direction

Site-to-Site VPN creates **two tunnels** for redundancy. If outbound from EC2 uses Tunnel 1 and the on-prem router responds via Tunnel 2, a stateful firewall on-prem that tracked the session on Tunnel 1 will drop the return traffic because it has no session state for Tunnel 2.

Fix: Configure BGP to prefer the same tunnel for both directions using AS path prepending or Local Preference. Or configure the on-prem firewall to be stateless for inter-tunnel flows.

Scenario B — EC2 has multiple ENIs or routes

If the EC2 instance has two ENIs (e.g., eth0 with a default route and eth1 connected to a VPC with VPN), the response traffic might egress via eth0 (internet) instead of eth1 (VPN path).

Fix: Add a specific route in the OS routing table:

```
ip route add 192.168.0.0/16 via 10.0.1.1 dev eth1
# where 192.168.0.0/16 is on-prem CIDR
```

Scenario C — VPN over Direct Connect (asymmetric paths)

If one direction uses Direct Connect and the other uses VPN as fallback, asymmetric routing will break stateful inspection.

Fix: Use BGP communities to ensure both directions prefer the same path. AWS supports BGP communities for DX to influence routing.

Diagnosis Steps:

1. Run `traceroute` from EC2 to on-prem IP and from on-prem to EC2 — compare hop paths.
2. Check CloudWatch VPN metrics: `TunnelState`, `TunnelDataOut` vs `TunnelDataIn` — if only one tunnel has bidirectional traffic, that's asymmetric.
3. On-prem: capture traffic on both tunnel interfaces to see which one carries return traffic.

Key Insights / Pitfalls

- **Most candidates forget about dual-tunnel VPN.** AWS always provisions two tunnels. If you're not doing active/passive BGP, both can be active and create asymmetry.
- **"Intermittent" is the tell.** Asymmetric routing problems are intermittent because ECMP (Equal Cost Multi-Path) alternates between paths — some flows work (same path both ways), some don't (different paths).
- **Never assume the firewall is stateless.** On-prem firewalls are almost always stateful. Asymmetric routing kills stateful inspection silently.

Scenario 1.3 — Service Fails After Adding a VPC Endpoint

> **Your application in a private subnet was successfully calling S3 through a NAT Gateway. A cost-optimisation task prompted you to add a Gateway VPC Endpoint for S3. After adding the endpoint, the application suddenly cannot reach S3. What happened?**

Answer

Adding a Gateway Endpoint for S3 modifies the **route table** — it inserts a route with destination `pl-xxxxxxxx` (S3's managed prefix list) pointing to the VPC endpoint. This route now takes **priority** over the `0.0.0.0/0` route to NAT Gateway for S3-bound traffic.

The likely causes of failure:

Cause 1 — Endpoint Policy is too restrictive

When you create a VPC Endpoint, it comes with a default policy that allows full access. But if someone attached a custom policy to the endpoint that restricts access to specific buckets or actions, the application's requests get denied.

```
// Example overly restrictive endpoint policy
{
  "Effect": "Allow",
  "Principal": "*",
  "Action": "s3:GetObject",
  "Resource": "arn:aws:s3:::specific-bucket/*"
}
```

If the app is writing to S3 (`s3:PutObject`) or accessing a different bucket, this blocks it.

Fix: Review and broaden the endpoint policy, or start with the AWS-managed default (`"Action": "s3:*"`).

Cause 2 — S3 Bucket Policy explicitly denies non-VPC-endpoint traffic (and now blocks legitimate access)

Some security-hardened bucket policies require requests to come through a specific VPC endpoint:

```
{
  "Effect": "Deny",
  "Principal": "*",
  "Action": "s3:*",
  "Resource": "arn:aws:s3:::my-bucket/*",
  "Condition": {
    "StringNotEquals": {
      "aws:sourceVpce": "vpce-aabbccdd"
    }
  }
}
```

If the endpoint ID in the bucket policy doesn't match the newly created endpoint, everything is denied.

Fix: Update the bucket policy condition to match the correct endpoint ID.

Cause 3 — The endpoint was added to the wrong route table

VPC Gateway Endpoints must be associated with the specific route table(s) of the subnets that need access. If you associated the endpoint with the public subnet route table but your app is in a private subnet, traffic still routes through NAT — and if the NAT's outbound path is now broken for other reasons, it fails.

Fix: Associate the endpoint with the **private subnet's route table**.

Cause 4 — Cross-region S3 access

Gateway endpoints only work **within the same region**. If your app accesses an S3 bucket in `us-west-2` but you're running in `us-east-1`, the endpoint won't handle cross-region traffic. That traffic must still go via NAT.

Fix: Keep the NAT Gateway for cross-region calls, use the endpoint only for same-region buckets.

Key Insights / Pitfalls

- **The endpoint silently intercepts traffic.** The route table change is invisible to the app — it has no idea the path changed. This is why failures feel mysterious.
- **Always review the endpoint policy after creation.** The default is permissive, but teams often paste in policies from documentation and forget to scope them correctly.
- **Cross-region S3 via VPC endpoint is not supported.** This catches a lot of people. If you're in `eu-west-1` and accessing `us-east-1` buckets, the endpoint won't help.
- **Test in staging first.** Route table changes are immediate and affect all traffic from that subnet.

Scenario 1.4 — Mysterious Latency Spike Between Two VPCs

> **You have two VPCs peered together. VPC-A is in `us-east-1` and VPC-B is also in `us-east-1`. Applications in VPC-A call a service in VPC-B. Latency is normally ~1ms but spikes to ~80ms every few minutes. No errors, just latency. How do you diagnose this?**

Answer

~80ms inter-region latency in what should be an intra-region peered connection screams **traffic is leaving the AWS backbone and taking an unintended path** — or there's resource contention.

Investigation Path:

Step 1 — Confirm traffic is actually going through the VPC peering connection

Check the route tables in both VPCs. VPC peering is not automatic — both VPCs need routes pointing to the peering connection (`pcx-xxxxxxxxx`) for the other's CIDR.

```
# If this route is missing, traffic falls back to 0.0.0.0/0 (NAT/IGW)
# and may end up going over the internet — explaining the 80ms spike
Destination: 10.1.0.0/16 → Target: pcx-xxxxxxxx
```

If the peering route is missing, traffic may be falling through to the default route and going out via Internet Gateway, then being routed back in — adding ~70–80ms of internet RTT.

Step 2 — Use VPC Reachability Analyser

AWS's Reachability Analyser traces the network path between two endpoints and shows exactly which hops are being used. Run it between the source EC2 in VPC-A and destination in VPC-B.

Step 3 — Check for DNS resolution issues

If the service in VPC-B is resolved by DNS to a public IP instead of private IP, traffic goes via the internet even if peering is set up correctly.

```
# From VPC-A instance, resolve the VPC-B service hostname
dig service.internal.vpce-b.example.com
# Should return 10.x.x.x (private)
# If it returns 52.x.x.x (public), DNS is wrong
```

VPC Peering does **not** automatically enable DNS resolution across VPCs. You must explicitly enable **"DNS resolution from remote VPC"** in the peering connection settings.

Step 4 — Correlate with CloudWatch and load patterns

If the route and DNS are correct, the intermittent spike may be resource contention:

- EC2 CPU credit exhaustion (T-series instances)
- ENI packet-per-second limits being hit
- Application-level connection pooling issues causing new TCP handshakes

Run `ping` between instances with timestamps during a spike. If ICMP also spikes, it's network. If ICMP is flat but app latency spikes, it's application-layer.

Key Insights / Pitfalls

- **The most dangerous mistake:** Assuming peering is configured correctly because "it was working before." Route tables and DNS settings can be modified or misconfigured later.
- **DNS is the most commonly missed piece of VPC peering.** AWS doesn't enable cross-VPC DNS resolution by default. You must check the peering connection settings in the console and enable it for both sides.
- **80ms is very specific.** That's US East Coast internet RTT. If you see exactly that range, traffic is going over the internet — not the backbone. That's a strong diagnostic clue.
- **Peering is not transitive.** If VPC-A → VPC-B → VPC-C, VPC-A cannot reach VPC-C through B. Attempting this means traffic hits VPC-B, finds no route to VPC-C, and may fall back to internet routing.

Scenario 1.5 — NAT Gateway Data Transfer Bill Is Unexpectedly High

> **Your team reviews the AWS bill and sees NAT Gateway data processing charges are 10x higher than expected. Your architecture has EC2 instances in private subnets calling AWS services like S3, DynamoDB, SSM, and CloudWatch. No architecture changes happened. What do you investigate?**

Answer

NAT Gateway charges $0.045 per GB processed. For AWS service calls, this is pure waste — traffic to AWS services should never leave the VPC.

Primary Cause — Missing VPC Endpoints for AWS services

Every API call to S3, DynamoDB, SSM, CloudWatch, ECR, Secrets Manager, etc., from a private subnet is being routed through NAT Gateway to the public AWS endpoints, generating data processing charges.

Fix — Add VPC Endpoints:

Service	Endpoint Type	Cost
S3	Gateway (route table)	**Free**
DynamoDB	Gateway (route table)	**Free**
SSM, SSM Messages	Interface (ENI)	~$7.30/endpoint/month
CloudWatch (monitoring, logs)	Interface (ENI)	~$7.30/endpoint/month
ECR API + DKR	Interface (ENI)	~$14.60/month
Secrets Manager	Interface (ENI)	~$7.30/endpoint/month

Prioritise: Add S3 and DynamoDB Gateway endpoints first — they're free and often account for the majority of NAT data volume, especially if S3 is used for application assets, logs, or Lambda deployment packages.

How to identify which services are generating traffic:

```
1. Enable VPC Flow Logs
2. Export to CloudWatch Logs Insights or Athena (S3)
3. Query for destination IPs belonging to AWS service ranges:
   - AWS publishes ip-ranges.json (https://ip-ranges.amazonaws.com/ip-
ranges.json)
   - Filter for service = "S3", "DYNAMODB", etc.
   - Correlate with your NAT Gateway ENI as the source
```

Secondary Cause — EC2 to EC2 traffic going via NAT

If two EC2 instances in the same VPC are communicating using **public IPs** instead of private IPs (e.g., a misconfigured config file or DNS returning public IP), traffic leaves via IGW, hits the internet, and returns — creating both NAT charges and data transfer charges.

Fix: Audit application configs and DNS to ensure inter-VPC and intra-VPC traffic uses private IPs.

Tertiary Cause — EC2 instances downloading packages via NAT

Large `yum update`, `apt-get upgrade`, or Docker pulls from DockerHub will all go through NAT. Consider:

- Using S3 for package caching (with S3 Gateway endpoint = free)
- Using ECR for Docker images (with ECR VPC endpoint)
- Using AWS CodeArtifact for package dependencies

Key Insights / Pitfalls

- **S3 and DynamoDB Gateway endpoints are free.** There is virtually no reason not to add them. This is low-hanging fruit that every SRE should know.
- **Candidates often forget that CloudWatch metrics and logs also flow through NAT.** In a heavily monitored fleet, this alone can be significant.
- **Interface endpoints have a cost (~$7.30/month/AZ).** Do a break-even analysis before adding them. If your NAT savings exceed the endpoint cost, add them.
- **Container workloads are high-volume offenders.** Every `docker pull` from DockerHub or ECR via NAT is expensive. Containerised environments need ECR VPC endpoints most urgently.

Section 2: EC2

Scenario 2.1 — The Instance That Won't Stay Healthy Behind an ALB

> **You register a new EC2 instance with an Application Load Balancer target group. The ALB console shows the instance as "unhealthy" within 30 seconds of registration. The instance appears to be running fine — you can SSH in and the application is running. What do you investigate?**

Answer

"Unhealthy" from ALB means the health check is failing. The ALB is making HTTP/HTTPS requests to the instance and not getting the expected response. SSH working proves the instance is alive — but the app layer may have issues.

Step 1 — Check the health check configuration

Go to the target group settings and verify:

- **Protocol:** HTTP or HTTPS? If the app only listens on HTTP but health check is HTTPS (or vice versa), it fails.
- **Port:** Health check port correct? Default is "traffic port" (e.g., 80), but if the app runs on 8080, it must be set explicitly.
- **Path:** Does the path return HTTP 200? If you configured `/health` but the app returns 404 on that path, it's unhealthy. If it returns a redirect (301/302), check whether the ALB is configured to accept 3xx codes.

- **Healthy threshold / Timeout:** Is the app slow to respond? If the health check timeout is 5s but the app takes 6s to warm up, it'll fail every check.

Step 2 — Check the Security Group on the instance

The ALB's health check originates from the ALB itself — specifically from the **ALB's Security Group**. The EC2's Security Group must allow inbound traffic on the health check port from the ALB's SG.

```
EC2 Security Group Inbound:
  Port: 80 (or app port)
  Source: sg-alb-xxxxxxx   ← ALB's Security Group ID
```

If someone locked the EC2 SG to specific IPs and forgot the ALB SG, health checks fail silently.

Step 3 — Test manually from the ALB's perspective

Use AWS Systems Manager Session Manager (no SSH needed) to run a curl from the instance itself:

```
curl -v http://localhost:80/health
```

If this fails, the application isn't responding correctly — investigate the app process, port binding, or startup errors in `/var/log/`.

If localhost works, try curling from the ALB subnet CIDR to simulate the ALB's health check source.

Step 4 — Check ALB access logs

Enable ALB access logs (written to S3). Health check requests will appear as requests from the ALB's IP. Look at the response code for health check paths.

Step 5 — Instance registration lag with slow-starting apps

If the app takes 60 seconds to fully start (e.g., Java app loading Spring context), the ALB starts health checking immediately on registration. The instance fails health checks and is marked unhealthy before it's ready.

Fix: Set a **registration delay** (deregistration delay doesn't help here) or use the **slow start mode** on the target group, which ramps up traffic gradually. Also increase the health check grace period if using Auto Scaling.

Key Insights / Pitfalls

- **Security Group source is a SG ID, not an IP range.** ALB IPs change. Always reference the ALB Security Group ID as the source — not a CIDR. This is the single most common cause of this exact problem.
- **HTTP 200 is the only default healthy response.** If your health endpoint returns 204 (No Content), you must explicitly add it to the acceptable response codes in the target group config.

- **Candidates forget about HTTPS health checks needing a valid cert.** If health check is HTTPS and the instance has a self-signed cert, the check may fail due to cert validation. Either use HTTP for health checks or configure the ALB to skip cert validation for them.
- **Deregistration delay ≠ registration delay.** These are different settings. Know which one prevents premature unhealthy marking.
- **ALB health check HealthyThresholdCount defaults to 5** — means 5 consecutive successes needed before instance is marked healthy. Reduce to 2 for faster warm-up in dev/staging.

Scenario 2.2 — Auto Scaling Group Is Launching and Terminating Instances in a Loop

> **Your Auto Scaling Group keeps launching new instances, marking them unhealthy, and terminating them in a continuous cycle. The instances terminate within 2–3 minutes of launch. CloudWatch shows scale-out events firing repeatedly. What is happening and how do you stop it?**

Answer

This is an **ASG health check death loop** — one of the nastiest production incidents to debug under pressure. The ASG launches instances, the health check marks them unhealthy before they're useful, ASG terminates them, and the process repeats.

Immediate mitigation — suspend health checks temporarily

```
aws autoscaling suspend-processes \
  --auto-scaling-group-name my-asg \
  --scaling-processes HealthCheck
```

This stops the termination loop while you diagnose. **Do not leave this suspended in production** — it disables self-healing.

Root Cause Investigation:

Cause 1 — Health check type is ELB, but ALB health checks are failing

If the ASG is configured with `HealthCheckType: ELB`, it trusts the ALB's health check. If the ALB marks the instance unhealthy (for reasons discussed in Scenario 2.1), ASG terminates it. But the instance might be functionally fine — the ALB health check might be misconfigured.

Fix: Fix the ALB health check first, then re-enable ASG health checks.

Cause 2 — Health check grace period is too short

The health check grace period is the time after launch during which ASG ignores health check failures. If your app takes 120 seconds to start but grace period is 30 seconds, the instance is checked before it's ready.

```
aws autoscaling update-auto-scaling-group \
  --auto-scaling-group-name my-asg \
  --health-check-grace-period 300
```

Cause 3 — User Data script is failing

If the `UserData` bootstrap script fails (e.g., a missing package, S3 access denied, bad environment variable), the app never starts. The instance comes up, passes the EC2 status check, but the ALB health check fails because the app isn't running.

Diagnosis:

```
# SSH into a new instance before it gets terminated, or use SSM
cat /var/log/cloud-init-output.log
# Look for errors in the UserData execution
```

Cause 4 — Scaling policy is over-sensitive

If a CloudWatch alarm triggers scale-out on a metric that immediately rises on new instance startup (e.g., CPU or connection count), more instances launch, which raises the metric further, triggering more launches.

Fix: Add cooldown periods to the scaling policy. Use step scaling instead of simple scaling. Or use target tracking policies which are inherently more stable.

Cause 5 — Instance store / EBS volume attachment failing

On some instance types with attached EBS volumes, if the volume attachment fails or takes too long, the app startup fails. Check EC2 system logs.

Key Insights / Pitfalls

- **Suspend-processes is your emergency brake.** Know this command before your interview and know which processes can be safely suspended (`HealthCheck`, `Launch`, `Terminate`).
- **Distinguish EC2 health check from ELB health check.** EC2 checks are basic (is the instance running?). ELB checks are application-level. ASG with ELB health check type is much stricter.
- **UserData failures are invisible without log access.** Candidates miss this. Always check `cloud-init-output.log` when instances aren't initialising correctly.
- **The loop has a cost impact.** Each instance that terminates was running for 2–3 minutes. At scale, you're burning significant spend. Know how to detect this via CloudWatch billing alerts.

Scenario 2.3 — EC2 Instance Suddenly Loses Network Connectivity Mid-Day

> **A production EC2 instance that has been running for weeks suddenly becomes unreachable from the internet. No deployments happened. SSH times out. The instance is showing**

> **"running" in the console and passes both status checks. Users report the app is down. What happened?**

Answer

The instance is alive (passes status checks) but unreachable — this strongly suggests a **networking configuration change**, not an instance failure.

Investigation Path:

Step 1 — Check Security Group changes

CloudTrail is your first stop. Search for recent `AuthorizeSecurityGroupIngress`, `RevokeSecurityGroupIngress`, or `ModifyInstanceAttribute` events.

```
aws cloudtrail lookup-events \
  --lookup-attributes
AttributeKey=EventName,AttributeValue=RevokeSecurityGroupIngress \
  --start-time 2024-01-01T00:00:00Z
```

A common scenario: someone ran a "security cleanup" script that revoked all broad inbound rules and forgot to add back the application port.

Step 2 — Check if the Elastic IP is still associated

If the instance used an EIP and it was accidentally disassociated (or associated to a different instance), the public IP is gone. The instance is reachable via private IP internally but not from the internet.

```
aws ec2 describe-addresses --filters "Name=instance-id,Values=i-xxxxxxxx"
```

Step 3 — Check the Route Table

Has the Internet Gateway route (`0.0.0.0/0 → igw-xxxxxxxx`) been accidentally removed from the public subnet route table? Without this, the subnet is effectively private.

Step 4 — Check the NACL

Someone may have added a DENY rule at a low rule number that now blocks all inbound traffic. NACLs evaluate rules in ascending order — a `DENY ALL` at rule 50 will override an `ALLOW 80` at rule 100.

Step 5 — VPC Flow Logs (if available)

```
If flow logs show REJECT on inbound → SG or NACL change
If flow logs show ACCEPT inbound but no outbound response → app crashed
internally (even though status checks pass)
If no flow log entries at all → traffic isn't reaching the ENI
(routing/EIP issue)
```

Step 6 — Use EC2 Serial Console or SSM for out-of-band access

If SSH is blocked by an SG/NACL change, use:

- **EC2 Serial Console** (if enabled on the account) — direct console access independent of networking.
- **AWS Systems Manager Session Manager** — doesn't require SSH or open port 22, uses the SSM agent.

If SSM Session Manager works but SSH doesn't, the issue is definitely network-level (SG/NACL/EIP), not the instance OS.

Key Insights / Pitfalls

- **Status checks passing does NOT mean the instance is reachable.** Status checks only verify the hypervisor and OS kernel — not that port 22 or 443 is accessible.
- **CloudTrail is your forensic tool.** Any network config change (SG modification, route table change, EIP disassociation) is logged. Always check CloudTrail before assuming it's an AWS fault.
- **NACL rule ordering surprises people.** A misconfigured NACL that adds DENY rules can silently kill production. Rules are evaluated lowest-number-first — a DENY at rule number 10 beats an ALLOW at rule 100.
- **SSM Session Manager is underutilised.** You don't need port 22 open at all for admin access if SSM Agent is installed. If your bastion goes down, SSM is your lifeline.

Scenario 2.4 — Spot Instance Interruption Is Crashing the Application

> **You migrated a stateless web tier to Spot Instances to save costs. Occasionally, users report 502 errors lasting about 2 minutes. You correlate these with Spot interruption notices. Your architecture uses an ASG and ALB. The interruptions are expected — but the 502s shouldn't last this long. What's wrong and how do you fix it?**

Answer

The 2-minute interruption warning window should give you enough time to gracefully drain the instance before it's terminated. The 502s indicate the instance is being killed before connections are fully drained. The problem is in the **deregistration and draining configuration**.

What should happen (correctly configured):

```
T-0:   Spot interruption notice arrives (2-minute warning)
       → EC2 sends interruption event via EventBridge
T-0:   EventBridge rule triggers Lambda/SSM to deregister instance from
ALB
T-0:   ALB begins connection draining (deregistration delay period)
T-~30s: All in-flight requests complete
T-2min: AWS terminates the instance (no active connections remain)
Result: Zero 502s
```

What's likely happening:

Problem 1 — Deregistration delay is too long or too short

ALB deregistration delay (default: 300 seconds) is how long ALB waits for in-flight requests to complete before forcibly cutting the connection. But the Spot warning is only 120 seconds. If deregistration delay > 120 seconds, ALB is still draining when AWS terminates the instance — any remaining requests get 502s.

Fix: Set deregistration delay to **90 seconds or less** on the target group (giving a 30-second buffer before the 120s termination):

```
aws elbv2 modify-target-group-attributes \
  --target-group-arn arn:aws:elasticloadbalancing:... \
  --attributes Key=deregistration_delay.timeout_seconds,Value=90
```

Problem 2 — Interruption notice is not triggering deregistration

AWS sends the interruption notice via the **instance metadata endpoint**, but the application or lifecycle hook must act on it. If nobody is polling the metadata endpoint or listening to EventBridge, the instance just terminates with no warning to the ALB.

Fix — Option A: EventBridge + Lambda

Create an EventBridge rule for `EC2 Spot Instance Interruption Warning` events that triggers a Lambda to deregister the instance from its target group.

Fix — Option B: ASG Lifecycle Hook

Add a lifecycle hook on `EC2_INSTANCE_TERMINATING`. When triggered:

1. The hook pauses termination for up to the heartbeat timeout.
2. A Lambda/SSM document runs to deregister from the ALB, drain connections, and complete the lifecycle action.

Fix — Option C: Instance polling

Run a lightweight agent that polls `http://169.254.169.254/latest/meta-data/spot/interruption-action` every 5 seconds. When a termination notice appears, deregister the instance and initiate graceful shutdown.

Problem 3 — New instance not ready in time

ASG launches a replacement instance, but it takes 60–90 seconds to become healthy. If the Spot instance terminates before the replacement is healthy, capacity drops and remaining instances are overloaded, causing 502s from timeouts.

Fix: Use **mixed instance type ASG policies** with at least 2 Spot instance pools (different instance families) to reduce simultaneous interruptions. Set minimum healthy percentage in the ASG to ensure replacements are pre-launched.

Key Insights / Pitfalls

- **The 2-minute warning is not guaranteed.** AWS best-effort sends it, but you cannot rely on always getting 120 seconds. Design for less.
- **Default deregistration delay of 300 seconds actively causes 502s on Spot.** This is a configuration mismatch almost every team makes initially.
- **EventBridge + Lambda is the cleanest solution.** Polling the metadata endpoint works but is fragile. EventBridge is the recommended AWS-native approach.
- **Don't use Spot for stateful workloads without careful thought.** Spot is ideal for stateless, idempotent processing. If your "stateless" tier writes to local disk during request processing, interruption will lose data.

Scenario 2.5 — CPU Is High but Application Isn't Doing Anything

> **An EC2 instance running your web application shows 90%+ CPU in CloudWatch. But when you SSH in, the application logs show very little activity. No deployments happened. What is consuming the CPU and how do you diagnose it?**

Answer

High CPU with low application activity is a classic "something else is running" or "the CPU metric is lying" scenario.

Step 1 — Get a real-time process view

```
top -c           # see all processes with command line args
# or
htop             # more visual, can sort by CPU
# Press 'P' in top to sort by CPU
```

Look for:

- **Unknown processes** consuming CPU — possible crypto miner if the instance was compromised.
- **System processes** spiking — kernel processes like `kswapd` (memory pressure), `kworker` (I/O), `softirq` (network interrupts).
- **Your application process** not appearing in logs but consuming CPU — possible stuck threads or GC thrashing.

Step 2 — Check CPU steal time

```
vmstat 1 10
# Look at the 'st' column (steal time)
```

If `%steal` is consistently > 5–10%, the **hypervisor is stealing CPU time** from your instance to give to other tenants. This is common on burstable T-series instances when burst credits are exhausted, or on oversubscribed hardware.

Fix: Move to a non-burstable instance type (M5, C5), or if on T-series, switch from Standard to Unlimited mode for the burst.

Step 3 — Check for T-series CPU credit exhaustion

```
# CloudWatch metric: CPUCreditBalance
# If this hits 0, the instance is throttled to baseline CPU
# Paradoxically, CloudWatch shows high CPU utilization of "baseline" CPU
# while the instance feels sluggish
```

Step 4 — Check for cryptomining or compromise

If you see processes like `xmrig`, `minerd`, `cgminer`, or unrecognised binary names in `/tmp` or `/var/tmp`:

```
# Check for unusual network connections
ss -tulnp
netstat -tulnp

# Check recently modified binaries
find /tmp /var/tmp /dev/shm -type f -newer /etc/passwd

# Check cron for injected jobs
crontab -l
cat /etc/cron*
```

Immediately isolate the instance by modifying its Security Group to deny all outbound except to your incident response systems.

Step 5 — Check for runaway application threads

For Java apps:

```
jstack <pid>    # thread dump — look for threads in RUNNABLE state stuck
in loops
jstat -gcutil <pid> 1000 10  # check GC activity — frequent full GCs =
memory pressure
```

For Node.js, Python, etc. — use language-specific profiling tools.

Step 6 — System-level I/O causing CPU wait

```
iostat -xz 1 5
# Look at %util and await for each disk
# High iowait in 'top' means CPU is waiting for disk, not actually
computing
```

Key Insights / Pitfalls

- **High CloudWatch CPU + low app activity = either stolen CPU or a rogue process.** These require completely different responses. Check steal time before assuming the application is the culprit.
- **T-series instances are a silent performance trap.** Teams deploy T3 instances for "cost savings" and then wonder why performance degrades under load. When credits exhaust, the instance is throttled. CloudWatch still shows "high CPU" at baseline utilisation.
- **Cryptomining on EC2 is a real incident type.** If an IAM key is leaked or an RCE vulnerability is exploited, attackers often deploy miners. The CPU spike is the first indicator. Know the forensics steps.
- **Never SSH into a potentially compromised instance from your workstation.** Isolate first, then investigate via SSM or a dedicated forensics instance.

Section 3: Databases

Scenario 3.1 — RDS Read Replica Is Falling Behind and Reads Are Stale

> **Your application uses an RDS MySQL primary with one read replica. The replica is used to offload reporting queries. Users report that their reports show data from 5–10 minutes ago, even though the primary has the latest data. The replica lag metric in CloudWatch is slowly climbing and sometimes reaches 600 seconds. What is causing this and how do you fix it?**

Answer

RDS replica lag (`ReplicaLag` in CloudWatch, or `Seconds_Behind_Master` in MySQL) is caused by the replica not being able to apply binlog events fast enough. MySQL replication is traditionally **single-threaded** for applying changes — a single large transaction on the primary can block all subsequent replication.

Root Cause Investigation:

Cause 1 — Long-running transactions or large batch writes on the primary

A single transaction that writes 2 million rows (e.g., a batch job, a bulk delete, a migration) generates a large binlog event. The replica must apply this entire transaction before processing anything after it. During this time, lag accumulates for all other changes.

Diagnosis:

```
-- On primary: identify long-running queries
SELECT * FROM information_schema.processlist
WHERE command != 'Sleep' AND time > 30
ORDER BY time DESC;

-- On replica: check replication status
```

```
SHOW SLAVE STATUS\G
-- Look at: Seconds_Behind_Master, Last_Error, Exec_Master_Log_Pos
```

Fix:

- Break large batch operations into smaller chunks (e.g., delete 10,000 rows at a time with a sleep between batches).
- Schedule heavy batch operations during off-peak hours.

Cause 2 — Single-threaded replication (MySQL 5.6 and earlier)

Traditional MySQL replication uses a single SQL thread to apply binlog events. If the primary is handling high concurrent write throughput, the replica can never keep up.

Fix: Enable **multi-threaded replication** (MySQL 5.7+):

```
-- On replica (RDS parameter group):
slave_parallel_workers = 4   -- or higher
slave_parallel_type = LOGICAL_CLOCK
```

In RDS, set this via a custom Parameter Group.

Cause 3 — Reporting queries blocking replication

Long-running `SELECT` queries on the replica can hold locks that block the replication SQL thread from applying updates.

Diagnosis:

```
SHOW PROCESSLIST;
-- Look for long-running SELECT queries on the replica
```

Fix:

- Set `max_execution_time` on reporting queries.
- Use `innodb_lock_wait_timeout` to limit lock wait time.
- Consider using RDS Aurora instead, which has near-zero replica lag due to its shared storage architecture.

Cause 4 — Replica instance is undersized

If the replica is a smaller instance type than the primary, it may not have enough CPU or I/O to keep up with replication while also serving reads.

Fix: Match replica instance type to primary, or upgrade. In RDS, you can resize the replica independently.

Mitigation for Immediate Relief:

If lag is critical, temporarily route reporting queries back to the primary (or pause them). Then fix the root cause.

For applications that need near-zero lag: migrate to **Amazon Aurora**, where replicas use the same shared storage as the primary — replica lag is typically under 100 milliseconds.

Key Insights / Pitfalls

- **Lag is cumulative.** A 60-second batch job that runs every 5 minutes means the replica is always 60 seconds behind minimum. Multiple such jobs compound.
- `Seconds_Behind_Master` **can be misleading.** It shows lag relative to the last processed binlog event, not real-time. If replication is completely stopped (`STOP SLAVE`), the metric stops updating — the lag could be infinite while the metric looks static.
- **Aurora replicas are not the same as RDS replicas.** Aurora replicas share the primary's storage and have lag under 100ms typically. RDS replicas replicate binlogs over network — fundamentally different architecture.
- **Read replicas are not for durability.** They are for read scaling only. For durability, use Multi-AZ. Candidates frequently conflate these two.

Scenario 3.2 — RDS Multi-AZ Failover Took 90 Seconds and Caused Application Errors

> **Your RDS MySQL instance is Multi-AZ enabled. During an AZ failure test, the failover took about 90 seconds. During that window, your application returned 500 errors to users. You expected Multi-AZ to be "transparent" to the application. Why wasn't it, and how do you reduce the impact?**

Answer

Multi-AZ RDS failover is **not instantaneous and not zero-downtime** — it typically takes 60–120 seconds. "Highly available" doesn't mean "zero-downtime." Understanding why and mitigating the impact is a core SRE responsibility.

Why Failover Takes 60–120 Seconds:

1. AWS detects the primary failure (health check polling interval: ~10s).
2. Promotes the standby to primary.
3. Updates the RDS endpoint DNS record to point to the new primary.
4. **DNS propagation** to clients — this is the main delay. The RDS endpoint hostname resolves to a new IP, but cached DNS entries must expire (TTL is typically 5 seconds for RDS endpoints, but clients may cache longer).

Why Your Application Had 500 Errors:

Problem 1 — Application's database connection pool holds stale connections

When failover happens, the old TCP connections to the former primary are severed. Connection pool libraries (HikariCP, SQLAlchemy, etc.) may hold onto these dead connections and attempt to reuse them for new queries — getting connection refused or timeout errors.

Fix: Configure the connection pool with:

```
# HikariCP example
connection-timeout: 3000        # fail fast on new connections
keepalive-time: 30000           # test connections proactively
max-lifetime: 1800000           # retire connections after 30 min
validation-timeout: 2000        # test before using
connection-test-query: SELECT 1 # optional but explicit
```

Problem 2 — Application has no retry logic for database errors

When a query fails due to a lost connection, the app immediately returns a 500 instead of retrying with exponential backoff.

Fix: Add retry logic for transient database exceptions:

```
from tenacity import retry, stop_after_attempt, wait_exponential

@retry(stop=stop_after_attempt(3),
       wait=wait_exponential(multiplier=1, min=1, max=10))
def execute_query(query):
    return db.execute(query)
```

Retry only on retriable errors (connection lost, deadlock). Do not retry non-retriable errors (constraint violations, syntax errors).

Problem 3 — DNS TTL caching in the JVM or application

Java has notoriously aggressive DNS caching. By default, JVMs may cache DNS lookups indefinitely. If the app caches the old IP, it keeps trying to connect to the dead primary.

Fix:

```
# For Java apps, set in JVM properties:
-Dsun.net.inetaddr.ttl=5
# Or in java.security:
networkaddress.cache.ttl=5
```

Problem 4 — Health checks kick out the app server before DB reconnects

If your app servers are behind an ALB and the DB failover makes them return 500s for 90 seconds, ALB health checks may mark the app servers unhealthy and remove them — compounding the outage.

Fix: Make health check endpoints resilient — they should return 200 even during brief DB unavailability (with a circuit breaker pattern).

Key Insights / Pitfalls

- **Multi-AZ is for durability (no data loss), not zero-downtime.** Candidates confuse this constantly. The standby is synchronously replicated but takes time to promote.
- **DNS TTL is the enemy of fast failover.** If your app or JVM aggressively caches DNS, failover takes longer from the application's perspective. Use connection validation, not just DNS refresh.
- **Aurora Multi-AZ has faster failover (~30s).** Aurora failover typically completes in 30 seconds. Aurora Global Database with read replicas can failover in under 1 minute cross-region.
- **Circuit breakers matter.** A correctly implemented circuit breaker opens on DB failure, immediately returns errors (or cached responses) without waiting for timeouts, and closes again once DB recovers. This prevents thread pool exhaustion during the 90-second window.

Scenario 3.3 — DynamoDB Throttling Despite Provisioned Capacity Looking Fine

> **Your DynamoDB table has 1,000 Write Capacity Units (WCUs) provisioned. CloudWatch shows average consumed WCUs at around 600 — well under the limit. But you're seeing `ProvisionedThroughputExceededException` errors and application latency spikes at peak times. How is this possible?**

Answer

This is the **hot partition problem** — one of the trickiest DynamoDB issues to diagnose. You have capacity to spare at the table level, but it's unevenly distributed across partitions, and one partition is being hammered.

How DynamoDB Partitioning Works:

DynamoDB distributes data across multiple partitions. Each partition has a capacity limit of **1,000 WCU and 3,000 RCU**. The table's total WCU (1,000 in your case) is spread across N partitions (e.g., 10 partitions at 100 WCU each). If all writes go to one partition, that partition hits its 100 WCU limit while the rest sit idle.

Identifying the Hot Partition:

Enable **CloudWatch Contributor Insights** for DynamoDB. This shows you:

- The most accessed partition keys.
- Which keys are consuming the most capacity.
- Throttled keys.

If one partition key or a small set of keys dominates, you have a hot partition.

Common Causes:

Cause 1 — Poor partition key choice

Using a low-cardinality key (e.g., `status = "active"`, `region = "us-east"`, `date = "2024-01-15"`) means all writes for that value go to the same partition.

Fix: Use a high-cardinality key (user_id, order_id, UUID) that distributes writes uniformly.

Cause 2 — Write sharding for intentionally hot keys

Sometimes a hot key is unavoidable (e.g., a "trending" item). You can shard writes by appending a random suffix to the partition key:

```
import random
# Instead of: partition_key = "product_123"
partition_key = f"product_123_{random.randint(1, 10)}"

# When reading, query all shards and merge:
for i in range(1, 11):
    query(f"product_123_{i}")
```

Cause 3 — Burst behaviour mismatched with provisioned capacity

DynamoDB allows you to burst up to **300 seconds of unused capacity** at a higher rate (burst capacity). If your workload is spiky, you may have burned through burst capacity and are now hitting hard limits, while the average still looks fine.

Fix: Use **DynamoDB Auto Scaling** or switch to **On-Demand capacity mode** if your workload is unpredictable. On-Demand has no partition limits (AWS manages it) but is more expensive per request.

Cause 4 — Global Secondary Index (GSI) is the bottleneck

If writes fan out to a GSI with a different partition key, the GSI might have hot partitions even if the base table doesn't. GSI throttling propagates back as write throttling on the base table.

Diagnosis: Check CloudWatch metrics separately for GSI: `ConsumedWriteCapacityUnits` on the GSI.

Key Insights / Pitfalls

- **Average capacity utilisation is misleading.** Always look at p99 and per-partition metrics, not averages. A table at 60% average can have one partition at 300%.
- **On-Demand mode does NOT eliminate hot partitions.** It helps with unpredictable traffic patterns but AWS still routes keys to partitions. A single continuously hot key can still throttle even in On-Demand mode — though the limits are higher.
- **DynamoDB Accelerator (DAX) helps with read hot partitions, not write hot partitions.** DAX is a caching layer — it absorbs repeated reads. It cannot help if writes to a single partition key are the problem.
- **Adaptive capacity is a partial mitigation.** DynamoDB does redistribute capacity automatically over time (adaptive capacity), but it's not instant — hot partition errors still occur during the rebalancing period.

Scenario 3.4 — Aurora Cluster Slows Down After a Blue-Green Deployment

> **You performed a blue-green deployment and switched traffic to a new Aurora cluster (the "green" environment). Within 10 minutes of the cutover, query latency doubles and the new cluster's `BufferCacheHitRatio` drops from 99% to 40%. The old cluster is still available. What happened and what do you do?**

Answer

The new Aurora cluster has a **cold buffer pool**. The old (blue) cluster had weeks or months of query patterns cached in its InnoDB buffer pool (in-memory page cache). The new (green) cluster has no warm cache — every query that expects data to be in memory is hitting disk instead, causing dramatically higher latency.

Why This Happens:

Aurora's buffer pool (InnoDB Buffer Pool) caches frequently accessed data pages in memory. On the blue cluster, the working set (hot data) was fully cached. On the green cluster, the buffer pool starts empty and must warm up — every cache miss is a disk read (even though Aurora uses SSD storage, disk I/O is orders of magnitude slower than memory access).

Immediate Mitigation:

Option 1 — Route back to blue cluster (rollback)

If the latency degradation is severe enough to impact users, route traffic back to the blue cluster immediately. The point of blue-green is this exact rollback capability.

```
# Update Route 53 CNAME or application config to point back to blue
endpoint
# Or update the ALB target group to blue cluster endpoint
```

Option 2 — Wait for buffer pool warm-up (if impact is tolerable)

The buffer pool warms up naturally as queries execute and cache misses populate it. This typically takes 15–30 minutes for a reasonably sized working set. Monitor `BufferCacheHitRatio` — as it climbs back towards 99%, latency will recover.

Option 3 — Pre-warm the buffer pool before cutover

Run representative queries against the green cluster before switching traffic:

```
# Example: run a representative workload against the green endpoint
# while still routing users to blue
SELECT SQL_NO_CACHE * FROM orders WHERE status = 'pending' LIMIT 10000;
SELECT SQL_NO_CACHE * FROM products WHERE category_id IN (...);
# etc. — target your most common queries
```

Option 4 — Aurora Fast Database Cloning

If using Aurora, you can clone the cluster — this shares the buffer pool warming benefit since Aurora clones use the same underlying storage pages. However, the in-memory buffer pool state is not transferred in a clone.

Long-term fix — Implement gradual traffic shifting:

Instead of a hard cutover, use **weighted routing in Route 53** or a feature flag to shift 5% → 20% → 50% → 100% of traffic to the green cluster over 30 minutes. This warms the cache under real traffic patterns before full cutover.

Key Insights / Pitfalls

- **Buffer pool warm-up is always a concern in database migrations and blue-green deployments.** It's a mandatory step that teams skip in their planning.
- `BufferCacheHitRatio` **is your leading indicator.** Set a CloudWatch alarm on this metric. If it drops below 90% during cutover, rollback automatically.
- **Aurora does not snapshot the buffer pool across clusters.** Unlike some databases that can dump and restore buffer pool state, Aurora doesn't have this built-in.
- **The symptom looks like under-provisioning.** Doubling of latency after a deployment makes people think the new cluster is under-resourced. It's not — it's cold. This confusion leads to unnecessary (and expensive) instance size upgrades.

Section 4: Security

Scenario 4.1 — IAM Role Assumed by the Wrong Service

> **You create an IAM role for an EC2 instance to read from an S3 bucket. The role is attached correctly and works fine. A week later, your security audit reveals that a Lambda function in the same account is also successfully assuming this role and accessing S3. You never intended to allow Lambda access. How did this happen and how do you prevent it?**

Answer

The trust policy on the IAM role is too permissive. The **trust policy** controls *who can assume the role* (the principal). If it's set to trust the entire AWS service `ec2.amazonaws.com` without further conditions, and if someone later manually called `sts:AssumeRole` from Lambda using a permissive execution role, they could assume it.

But more likely: the Lambda function's **execution role** has `sts:AssumeRole` permissions on `*` (all roles), and someone explicitly added the Lambda's role as a principal in the EC2 role's trust policy by mistake, or the trust policy uses an overly broad condition.

Diagnosis:

Step 1 — Examine the role's trust policy:

```
{
  "Effect": "Allow",
  "Principal": {
    "Service": "ec2.amazonaws.com"
  },
```

```
    "Action": "sts:AssumeRole"
  }
```

If this is the only trust statement, Lambda *cannot* directly assume this role. Lambda would need to be explicitly listed.

Step 2 — Check CloudTrail for `AssumeRole` events:

```
aws cloudtrail lookup-events \
  --lookup-attributes AttributeKey=EventName,AttributeValue=AssumeRole \
  --start-time 2024-01-01T00:00:00Z \
  | jq '.Events[] | select(.CloudTrailEvent | fromjson |
.requestParameters.roleArn | contains("your-ec2-role"))'
```

This reveals *who* assumed the role and *when*.

Step 3 — Check the Lambda execution role's permissions: If the Lambda role has `sts:AssumeRole` on `*`, it can assume any role in the account that trusts it.

How to Fix and Prevent:

Fix 1 — Add `aws:SourceAccount` and `aws:PrincipalArn` conditions to the trust policy:

```
{
  "Effect": "Allow",
  "Principal": {
    "Service": "ec2.amazonaws.com"
  },
  "Action": "sts:AssumeRole",
  "Condition": {
    "StringEquals": {
      "aws:SourceAccount": "123456789012"
    }
  }
}
```

Fix 2 — Use Permission Boundaries on the Lambda execution role: A permission boundary caps what the Lambda role can do, even if it has `sts:AssumeRole` — the boundary can deny assuming roles intended for other services.

Fix 3 — Implement IAM Access Analyser: Enable **IAM Access Analyser** at the organisation level. It flags roles that can be assumed by principals outside their intended scope, including cross-service assumptions.

Fix 4 — Use `aws:CalledVia` and `ec2:SourceInstanceARN` conditions (where supported): For fine-grained control, some AWS services support conditions that verify the exact calling service.

Key Insights / Pitfalls

- **Trust policies and permission policies are separate and both must be correct.** A common misconception is that attaching a role to EC2 makes it exclusive to EC2. The trust policy controls assumption rights — if another principal satisfies the trust policy, it can assume the role.
- `sts:AssumeRole` **on** `*` **in any role's permissions policy is a red flag.** Audit for this during security reviews. A role with this permission can potentially escalate privileges by assuming more permissive roles.
- **IAM Access Analyser is underutilised.** It automates exactly this kind of analysis — flagging unintended cross-principal access. Every AWS account should have it enabled.
- **The Confused Deputy problem.** When one service (Lambda) can trick another service (STS) into performing actions using a role intended for a different service (EC2), it's a confused deputy attack. AWS addresses this with source conditions in trust policies.

Scenario 4.2 — CloudTrail Shows an IAM Key Making API Calls from an Unusual IP

> **CloudTrail shows API calls being made with an IAM access key from an IP address in Eastern Europe. Your team is based in India. The calls are listing S3 buckets and describing EC2 instances. You believe the key may have been leaked (possibly committed to a public GitHub repo). Walk through your incident response step by step.**

Answer

This is a **credential compromise incident**. Speed and decisiveness matter — every minute of delay is another minute the attacker has access.

Phase 1 — Immediate Containment (first 5 minutes)

Step 1 — Disable the compromised key immediately:

```
aws iam update-access-key \
  --access-key-id AKIAIOSFODNN7EXAMPLE \
  --status Inactive \
  --user-name compromised-user
```

Do NOT delete yet — disabling preserves forensic information and can be quickly reversed if it's a false positive. Deleting is irreversible.

Step 2 — Revoke all active sessions for the associated IAM user:

```
aws iam attach-user-policy \
  --user-name compromised-user \
  --policy-arn arn:aws:iam::aws:policy/AWSDenyAll
```

This denies all actions for this user regardless of other policies, effectively shutting down the user immediately.

If it's an IAM role (not a user), you cannot revoke existing sessions directly — but you can add a **deny policy with an AWSRevokeOlderSessions condition**:

```
{
  "Effect": "Deny",
  "Action": "*",
  "Resource": "*",
  "Condition": {
    "DateLessThan": {
      "aws:TokenIssueTime": "2024-01-15T10:30:00Z"
    }
  }
}
```

Any session token issued before this timestamp is now denied.

Phase 2 — Blast Radius Assessment (next 30 minutes)

Step 3 — Determine what the attacker accessed or modified:

```
# Search CloudTrail for all actions by this key in the past 90 days
aws cloudtrail lookup-events \
  --lookup-attributes
AttributeKey=AccessKeyId,AttributeValue=AKIAIOSFODNN7EXAMPLE \
  --max-results 50

# Check for: new IAM users/keys created, S3 GetObject (data exfiltration),
# EC2 instances launched (crypto mining), Lambda functions created,
# Security Group modifications, CloudTrail trail modifications (covering
tracks)
```

High-risk actions to look for:

- `CreateUser` / `CreateAccessKey` — attacker may have created a backdoor account.
- `PutBucketPolicy` or `GetObject` — data may have been exfiltrated.
- `RunInstances` — crypto miners may have been launched.
- `DeleteTrail` or `StopLogging` — attacker may have tried to cover tracks.

Step 4 — Check for attacker-created backdoors:

```
# List all IAM users and look for recently created accounts
aws iam list-users --query 'Users[*].[UserName,CreateDate]' --output table

# List all access keys across all users
aws iam generate-credential-report
aws iam get-credential-report
```

Phase 3 — Remediation

Step 5 — Remove the compromised key from GitHub/source control:

```
git filter-branch --force --index-filter \
  'git rm --cached --ignore-unmatch path/to/config/file' \
  --prune-empty --tag-name-filter cat -- --all
git push origin --force --all
```

Also invalidate the key in AWS (already done in Step 1) and rotate any secrets stored near it.

Step 6 — Terminate attacker-created resources: Any EC2 instances, Lambda functions, or IAM users created by the attacker should be identified and removed. Do not do this before capturing forensic data (Step 3).

Step 7 — Enable AWS GuardDuty if not already active: GuardDuty would have flagged `UnauthorizedAccess:IAMUser/InstanceCredentialExfiltration` or `Recon:IAMUser/MaliciousIPCaller` automatically. Enable it so this is detected proactively next time.

Phase 4 — Post-Incident Hardening

- Enable **GitHub secret scanning** to detect future key commits before they go public.
- Implement **AWS Config rules** to alert when access keys are older than 90 days.
- Apply **SCP (Service Control Policy)** at the org level to restrict `iam:CreateUser` to specific roles.
- Require **MFA** for all IAM users and enforce it via IAM policy conditions.
- Use **IAM roles for EC2/Lambda instead of long-term access keys** wherever possible.

Key Insights / Pitfalls

- **Disable before delete.** Disabling preserves forensic data (creation date, last used, etc.). Deleting is permanent and loses that context. Disable first, delete after investigation.
- **CloudTrail has a 90-day lookup limit via the console.** For deeper historical analysis, you need CloudTrail logs in S3 and Athena for querying. Proactive log delivery to S3 is non-negotiable for security.
- **Assume the attacker created a backdoor.** The `list-buckets` and `describe-instances` calls may be reconnaissance before deeper activity. Always check for newly created IAM users/keys.
- **GuardDuty is the difference between minutes and days to detection.** It analyses CloudTrail, VPC Flow Logs, and DNS logs automatically and fires findings in near-real-time. Manual CloudTrail review after the fact is always retrospective.

Scenario 4.3 — S3 Bucket Data Is Exposed to the Public Despite a Bucket Policy Denying Public Access

> **Your S3 bucket has a bucket policy that explicitly denies `s3:GetObject` to everyone except your application role. S3 Block Public Access is NOT enabled at the bucket level (it was turned off during a migration and never re-enabled). A security scanner reports that certain objects in**

> **the bucket are publicly accessible. You look at the bucket policy and it looks correct. What is happening?**

Answer

The **bucket policy** denies public access — but that's not the only mechanism that can grant access. S3 access control is layered, and an **Access Control List (ACL)** on individual objects may be granting public access despite the bucket policy.

S3 Access Evaluation Order:

AWS evaluates S3 access in this order:

1. **Block Public Access settings** (account-level and bucket-level) — if enabled, they override everything else.
2. **IAM policies** (for authenticated requests).
3. **Bucket policies**.
4. **Object ACLs** (if the bucket owner enforces object ownership is not enforced).

An explicit `Deny` in a bucket policy overrides an `Allow` in an IAM policy — **but ACLs and bucket policies are evaluated independently for public access.** If the object ACL grants `public-read`, *and* there's no Block Public Access enabled, the object is accessible publicly even if the bucket policy has a Deny.

Root Cause:

During the migration, objects were likely uploaded with a **public-read ACL**:

```
aws s3 cp file.txt s3://my-bucket/ --acl public-read
```

Or the bucket's default ACL was set to public-read during the migration, applying to all newly uploaded objects.

How to Identify Affected Objects:

```
# List all objects and check their ACLs
aws s3api list-objects-v2 --bucket my-bucket --query 'Contents[*].Key' --
output text | \
  xargs -I{} aws s3api get-object-acl --bucket my-bucket --key {}

# Or use S3 Inventory (for large buckets) with ACL metadata included
# Configure an S3 Inventory report to include ACL data
```

Fix:

Fix 1 — Enable Block Public Access immediately:

```
aws s3api put-public-access-block \
  --bucket my-bucket \
  --public-access-block-configuration \

"BlockPublicAcls=true,IgnorePublicAcls=true,BlockPublicPolicy=true,Restric
tPublicBuckets=true"
```

`IgnorePublicAcls=true` makes AWS **ignore all public ACLs** — objects with `public-read` ACL are no longer publicly accessible. This is the fastest remediation.

Fix 2 — Remove public ACLs from all objects:

```
# Reset all object ACLs to private (do this after enabling BPA to be safe)
aws s3api list-objects-v2 --bucket my-bucket --query 'Contents[*].Key' --
output text | \
  tr '\t' '\n' | \
  xargs -P 10 -I {} aws s3api put-object-acl --bucket my-bucket --key {} -
-acl private
```

Fix 3 — Enable S3 Object Ownership with "Bucket owner enforced": This disables ACLs entirely for the bucket. All objects are owned by the bucket owner. Access is controlled exclusively via bucket policies.

```
aws s3api put-bucket-ownership-controls \
  --bucket my-bucket \
  --ownership-controls 'Rules=[{ObjectOwnership=BucketOwnerEnforced}]'
```

Key Insights / Pitfalls

- **Bucket policy `Deny` does not override object ACLs for public access.** This surprises almost everyone. The two mechanisms are independent when it comes to unauthenticated (public) requests.
- **Block Public Access is the safest default.** Enable it at the AWS account level so it applies to all new buckets by default. This is now the AWS default for new accounts.
- **ACLs are a legacy feature.** AWS recommends disabling ACLs via `BucketOwnerEnforced` ownership and using bucket policies exclusively. Fewer mechanisms = less confusion.
- **During migrations, temporary permissive settings get forgotten.** "We'll fix it after the migration" is how security debt accumulates. Post-migration checklists must include re-enabling security controls.

Scenario 4.4 — An EC2 Instance Is Making Unexpected API Calls to AWS Services

> **GuardDuty fires an alert:**
> `UnauthorizedAccess:IAMUser/InstanceCredentialExfiltration`**. An EC2 instance's**

> **IAM role credentials are being used from an IP address that is not the instance's IP. The instance is running a public-facing web application. What happened and what do you do?**

Answer

The finding `InstanceCredentialExfiltration` means credentials retrieved from the **EC2 Instance Metadata Service (IMDS)** are being used outside of the instance. This is a **Server-Side Request Forgery (SSRF) attack** leading to credential theft.

How the Attack Worked:

1. The attacker found an SSRF vulnerability in the web application (e.g., an endpoint that fetches a URL provided by the user).
2. The attacker crafted a request like:

   ```
   GET /fetch?url=http://169.254.169.254/latest/meta-data/iam/security-credentials/
   ```

3. The application fetched this URL internally and returned the response to the attacker.
4. The attacker used the returned `AccessKeyId`, `SecretAccessKey`, and `SessionToken` from outside AWS to make API calls.

Why This Works (with IMDSv1):

IMDSv1 is a simple HTTP GET to `169.254.169.254`. Any process that can make HTTP requests can access it — including a web application acting as an SSRF proxy. There is no authentication required.

Immediate Response:

Step 1 — Revoke the instance's role credentials:

Since these are temporary credentials from STS, add a deny-all policy with a timestamp condition to the IAM role (as described in Scenario 4.2, Phase 1). This invalidates all credentials issued before "now."

Step 2 — Isolate the EC2 instance:

```
# Assign a restricted Security Group that blocks all inbound/outbound
# except to your forensics/IR systems
aws ec2 modify-instance-attribute \
  --instance-id i-xxxxxxxx \
  --groups sg-forensics-only
```

Step 3 — Capture forensic data before termination:

- Create an EBS snapshot of the root volume.
- Dump memory if possible (using SSM or a pre-installed agent).
- Pull application logs from CloudWatch Logs.

Step 4 — Check CloudTrail for actions taken with the stolen credentials: Same process as Scenario 4.2 — look for data exfiltration, new IAM users, compute launches.

Remediation — Upgrade to IMDSv2:

IMDSv2 requires a **session-oriented flow** with a PUT request to get a token before making metadata requests. This breaks most SSRF attacks because:

- SSRF vulnerabilities typically only follow GET redirects or make GET requests.
- IMDSv2's first step requires a PUT with a specific header (`X-aws-ec2-metadata-token-ttl-seconds`).
- Protocols that don't support arbitrary headers or PUT requests cannot retrieve the token.

Enforce IMDSv2:

```
# Require IMDSv2 on an existing instance
aws ec2 modify-instance-metadata-options \
  --instance-id i-xxxxxxxx \
  --http-tokens required \
  --http-endpoint enabled

# Enforce IMDSv2 for all new instances via AWS Config or Launch Template
```

Enforce via Launch Template:

```
{
  "MetadataOptions": {
    "HttpTokens": "required",
    "HttpEndpoint": "enabled",
    "HttpPutResponseHopLimit": 1
  }
}
```

`HttpPutResponseHopLimit: 1` ensures the token response doesn't travel beyond the instance — a container cannot use the host instance's metadata credentials.

Fix the SSRF Vulnerability:

- Validate and whitelist URLs your application is allowed to fetch.
- Block requests to `169.254.169.254` and link-local addresses at the application level.
- Apply Web Application Firewall (WAF) rules to detect SSRF patterns.

Key Insights / Pitfalls

- **IMDSv1 should be disabled on all production instances.** It is an SSRF enabler by design. IMDSv2 is the mitigation, not a nice-to-have.
- **`HttpPutResponseHopLimit: 1` is critical for containers.** Without it, a container can make a request that hops through the host network stack and reaches the host's metadata endpoint, stealing

the host's IAM role credentials.

- **GuardDuty fires this finding in near-real-time.** Without GuardDuty, this attack could go undetected for days. The attacker would have credentials valid for up to 6 hours (the default STS session duration for instance roles).
- **Temporary credentials don't mean less danger.** A 6-hour window with an overly permissive IAM role can cause enormous damage — data exfiltration, resource creation, persistence via new IAM keys.

Scenario 4.5 — KMS Key Deletion Is Causing Cascading Failures Across Services

> **A junior engineer ran `aws kms schedule-key-deletion` on a KMS key, setting the pending deletion window to 7 days. Within hours, multiple services start failing: RDS can't read encrypted snapshots, S3 objects with SSE-KMS return `KMS.DisabledException`, and EBS volumes fail to mount on new instances. What's happening and how do you recover?**

Answer

`schedule-key-deletion` doesn't immediately delete the key — it **disables** the key immediately and schedules permanent deletion after the pending window (7–30 days). Even though the key still exists, it's in a `PendingDeletion` state and **all cryptographic operations are immediately rejected**. Any data encrypted with this key becomes inaccessible.

What's Happening to Each Service:

- **RDS snapshots:** Encrypted with this KMS key. Restore attempts call KMS to decrypt the storage encryption key — rejected.
- **S3 SSE-KMS objects:** Every `GetObject` call requires a KMS `Decrypt` call to decrypt the data key — rejected.
- **EBS volumes:** When EC2 attempts to mount an encrypted EBS volume, it calls KMS to decrypt the volume encryption key — rejected. Existing mounted volumes may stay accessible (the data key is cached in memory) but new mounts fail.

Immediate Recovery Step — Cancel the Key Deletion:

```
aws kms cancel-key-deletion --key-id arn:aws:kms:us-east-
1:123456789012:key/xxxxxxxx
```

This re-enables the key immediately. **You have up to the end of the pending deletion window to cancel.** Once the window expires and the key is permanently deleted, data encrypted with it is irreversibly lost — forever.

After cancellation, re-enable the key:

```
aws kms enable-key --key-id arn:aws:kms:us-east-
1:123456789012:key/xxxxxxxx
```

Services should automatically resume within minutes as they retry KMS operations.

Why Services Don't Warn You Adequately:

KMS key deletion has a mandatory minimum waiting period (7 days) specifically to allow this recovery. AWS designed this as a safeguard because there is no recovery after permanent deletion. The cascading failures are the "warning" — AWS cannot warn you ahead of time because it doesn't know what data you've encrypted with a given key.

How to Prevent This:

Prevention 1 — IAM and SCP controls on `kms:ScheduleKeyDeletion`:

```
{
  "Effect": "Deny",
  "Action": "kms:ScheduleKeyDeletion",
  "Resource": "*",
  "Condition": {
    "StringNotEquals": {
      "aws:PrincipalArn":
"arn:aws:iam::123456789012:role/SecurityAdminRole"
    }
  }
}
```

Only a specific security admin role can schedule key deletions.

Prevention 2 — AWS Config rule to detect key deletion scheduling: Create an AWS Config rule that fires whenever a KMS key enters `PendingDeletion` state and sends an SNS alert.

Prevention 3 — Key aliases for operational abstraction: Use KMS key aliases (`alias/production-rds-key`) instead of key ARNs directly in service configurations. This way, you can rotate the underlying key by updating the alias without changing configurations — and it makes it clearer which key is critical.

Prevention 4 — Key usage reports before deletion: Before deleting any KMS key, run an audit:

```
# Check CloudTrail for recent key usage
aws cloudtrail lookup-events \
  --lookup-attributes
AttributeKey=ResourceName,AttributeValue=arn:aws:kms:...:key/xxx \
  --start-time 2024-01-01T00:00:00Z

# Check AWS Config for resources encrypted with this key
# (or use the KMS key's CloudTrail events to identify which services used
it)
```

Key Insights / Pitfalls

- `schedule-key-deletion` **disables the key immediately, not after the waiting period.** The waiting period is time to cancel — not a grace period before impact. This is the most important and most misunderstood aspect.
- **Permanent KMS key deletion = permanent data loss.** There is no AWS support escalation that can recover data from a deleted KMS key. The waiting period (7–30 days) is the only safeguard.
- **EBS volumes may not fail immediately.** Running instances have the data encryption key cached in memory. But stop/start or new instance attachments will fail. This creates confusing intermittent behaviour.
- **The minimum 7-day deletion window is AWS's safety net.** If someone accidentally deletes a key, you have at least 7 days to detect and recover. This is why extending the window to 30 days is a security best practice.

End of AWS Scenario-Based Interview Prep

> **Revision Notes:**
>
> - Re-read each scenario and mentally trace the architecture before reading the answer.
> - For SRE roles: focus on diagnosis methodology — interviewers want to see structured thinking under pressure.
> - For Cloud Engineer roles: emphasise the fix steps and the "why" behind each configuration choice.
> - For DevOps roles: tie answers back to automation — how would you prevent this with IaC, CI/CD, and GitOps patterns?